

Hagzy SuperAdmin

Platform Management Dashboard

Comprehensive Technical & Operational Documentation

Version 0.1.0 | April 2026

Prepared for developers, operations, and stakeholders

Table of Contents

- 1. Executive Summary 7**
 - At a Glance 7
- 2. Stakeholder Overview 8**
 - 2.1 What the Dashboard Does 8
 - 2.2 Who Uses It 8
 - 2.3 Why Each Module Exists 9
 - 2.4 Key Workflows (End-to-End) 9
 - 2.4.1 New Provider Onboarding 9
 - 2.4.2 Subscription Sale + Renewal 9
 - 2.4.3 Weekly Finance Close 10
 - 2.4.4 Review Moderation 10
 - 2.4.5 Incident Handling 10
- 3. Tech Stack & Dependencies 11**
- 4. Architecture 12**
 - 4.1 High-Level Architecture 12
 - 4.2 Rendering Model 12
 - 4.3 Folder Layout 13
 - 4.4 Data Flow 14
 - 4.5 Provider Tree 14
- 5. Authentication & Authorization 15**
 - 5.1 Login Flow 15
 - 5.2 Public vs Protected Routes 15
 - 5.3 RBAC: The Permission Matrix 15
 - 5.4 Permission Gates in Code 16
 - 5.5 Impersonation 16
- 6. Complete Route Map 17**
- 7. Module Deep Dive 19**
 - 7.1 Dashboard (/) 19
 - 7.2 Providers 19
 - /providers 19

/providers/[id]	19
/providers/registrations	19
7.3 Users & Wallets	19
/users	19
/users/wallets	20
7.4 Subscriptions	20
/subscriptions	20
/subscriptions/plans	20
/subscriptions/invoices	20
7.5 Finance	20
Commission states	20
Payout states	20
7.6 Reviews	21
7.7 Marketing	21
/marketing/campaigns	21
/marketing/ads & /marketing/ads/experiments	21
/marketing/promo-codes	21
/marketing/push-notifications	21
7.8 Content	21
7.9 Reports	21
7.10 Support	22
7.11 App Versions	22
7.12 System Health	22
7.13 Audit Logs	22
7.14 Settings	22
/settings/appearance	22
/settings/admins (admin only)	22
/settings/roles (admin only)	22
/settings/security (admin only)	22
/settings/payments	23
/settings/countries	23
/settings/platform	23
/settings/localization	23

8. Component Reference 24

8.1 Admin Components	24
----------------------	----

8.2 Layout Components	24
8.3 Tables	24
8.4 Design-System UI Library	25
8.5 Top-Level Utility Components	26
9. Hooks Reference	27
9.1 Hook Usage Examples	28
useValidatedForm	28
useApiQuery + DataGuard	28
useFocusTrap	28
10. Context Providers	29
10.1 AuthContext	29
10.2 ThemeContext	29
10.3 LanguageContext	29
10.4 NotificationsContext	29
10.5 SidebarContext	29
11. Type Reference	30
11.1 Admin & Permissions (admin.ts)	30
11.2 Providers (provider.ts)	31
11.3 Users & Wallets (user.ts, wallet.ts)	31
11.4 Subscriptions (subscription.ts)	32
11.5 Finance & Marketing & Content	33
11.6 System (system.ts) & Support (ticket.ts)	34
12. Library Reference (src/lib)	35
12.1 api.ts	35
12.2 permissions.ts	35
12.3 utils.ts	35
12.4 validations.ts	35
12.5 analytics.ts	36
13. Styling, Theming & i18n	37
13.1 Styling	37
13.2 Theming	37
13.3 Internationalization	37

13.4 Responsive Breakpoints	38
13.5 Accessibility Summary	38
14. Mock Data Layer	39
15. End-User Guide	40
15.1 Signing In	40
15.2 Using the Command Palette	40
15.3 Filters, Search, and Sort	40
15.4 Exporting Data	40
15.5 Approving a Provider	40
15.6 Adjusting a User's Wallet	40
15.7 Processing Payouts	41
15.8 Moderating a Review	41
15.9 Sending a Push Notification	41
15.10 Responding to a Support Ticket	41
15.11 Releasing a Mobile App Version	41
16. Developer Onboarding	42
16.1 Running Locally	42
16.2 Project Conventions	42
16.3 Adding a New Page	43
16.4 Adding a New Module Permission	43
16.5 Adding a New Zod Schema	43
16.6 Git Workflow (Suggested)	43
17. Testing Strategy	45
18. Deployment	46
18.1 Recommended: Vercel	46
18.2 Self-hosted	46
18.3 Environment Variables	46
18.4 Pre-release Checklist	46
19. Analytics Heuristics (Reference)	47
19.1 Provider Performance Score	47
19.2 Churn Risk	47
19.3 Revenue Forecast	47

19.4 A/B Test	47
19.5 Anomaly Detection	48
20. Glossary	49
21. Appendix A — File Index	50
22. Appendix B — Future Work	52

1. Executive Summary

Hagzy SuperAdmin is the operational control plane for the Hagzy booking platform — a multi-tenant marketplace connecting end users with service providers across beauty, wellness, and health categories (salons, barbers, clinics, spas, nail studios, and more). This dashboard is the single pane of glass that Hagzy staff use to onboard providers, govern subscriptions, moderate content, process payouts, coordinate support, and analyze platform health in real time.

The application is built with **Next.js 16.2** (App Router) and **React 19.2**, using TypeScript end-to-end. It ships over **50 routes** organized into 14 functional modules, protected by a fully configurable role-based access control (RBAC) system with **six roles, 18 permission modules**, and **six actions** per module. The UI is fully responsive, supports English/Arabic with right-to-left layout, and ships with light, dark, and system theme modes. A custom service worker gives the app a graceful offline experience, and accessibility features such as focus traps, live regions, and command-palette keyboard navigation make the product usable for power users and assistive-technology users alike.

This document is the single source of truth for Hagzy SuperAdmin. It is written for three audiences: engineers extending the codebase, operations staff using the dashboard day to day, and stakeholders evaluating capabilities or planning roadmap. Use the table of contents to jump to the section that matches your role.

At a Glance

Attribute	Value
Product	Hagzy SuperAdmin Dashboard
Version	0.1.0
Framework	Next.js 16.2.3 (App Router)
Language	TypeScript 5 · React 19.2.4
Routes	50+ across 14 modules
Permission model	6 roles × 18 modules × 6 actions = 648 permission cells
Localization	English, Arabic (RTL)
Theming	Light / Dark / System
Data layer	Mock data at src/mocks/ (API-ready via src/lib/api.ts)
State	React Context + custom hooks (no Redux/Zustand)
Forms	react-hook-form + Zod validation
Charts	Recharts (Area, Pie, Bar, Line)
Offline	Service worker with online/offline detection
Dev port	3001 (next dev)

2. Stakeholder Overview

This section distills the product for non-technical readers: what the dashboard does, who operates it, and why each module exists. It is the recommended reading order for executives, investors, and new-hires before diving into the engineering sections.

2.1 What the Dashboard Does

Hagzy SuperAdmin lets Hagzy staff run the platform. Every user-visible surface — mobile app, provider app, web site, email, SMS, push — has a corresponding configuration or moderation surface in the dashboard. A single operator can approve a new salon in the morning, adjust the commission rate at noon, push a marketing campaign after lunch, review flagged customer reviews in the afternoon, reconcile payouts at close of business, and audit a sensitive action before heading home — all without leaving the browser.

2.2 Who Uses It

Role	Responsibility	Typical head-count
Super Admin	Unrestricted. Configures roles, payment gateways, platform settings, impersonates providers.	2-4
Admin	Full CRUD across all modules except impersonation. Day-to-day operations lead.	5-15
Moderator	Reviews, content, marketing, and ads moderation. Read-only for providers/users.	3-10
Support	Support ticket workflow, wallet visibility, read-only on users/providers/reviews.	5-30
Finance	Read-only providers. Full CRUD + export on subscriptions and finance. Read + export on reports.	2-6
Viewer	Read-only everything. Auditors, external consultants, read-only QA.	unlimited

2.3 Why Each Module Exists

Module	Purpose
Dashboard	Daily KPI check — revenue, bookings, churn-risk providers, recent activity.
Providers	Lifecycle of service providers: registration review, profile, branches, performance scoring, impersonation.
Users	Lifecycle of end-users and their wallets: status, balance, refunds, adjustments.
Subscriptions	Selling, billing, and renewing provider plans. Plan catalog, invoices, trials.
Finance	Revenue recognition — commissions, payouts, tax reports, forecasting.
Reviews	Quality and trust — moderation queue, analytics, admin responses.
Marketing	Growth — campaigns, banners, promo codes, featured listings, push notifications, A/B ads.
Content	User-facing copy — announcements, legal pages, categories, email/SMS templates.
Reports	Analytics deep dive — revenue, bookings, users, providers, geographic.
Support	Ticket queue with SLA and assignment.
App Versions	Mobile release management — force-update toggles, release notes.
System Health	Real-time service status board.
Audit Logs	Forensic trail with anomaly detection (burst, off-hours, high-risk, reversal).
Settings	Platform-wide configuration — admins, roles, security, countries, payments, appearance, localization.

2.4 Key Workflows (End-to-End)

2.4.1 New Provider Onboarding

A salon applies through the provider mobile app. The request lands in **Providers** → **Registrations** with status *pending*. The operator reviews uploaded documents, approves with optional edits (name, category, commission), or rejects with a reason. On approval the provider becomes active on the platform and appears in **Providers**. Audit log captures the approval.

2.4.2 Subscription Sale + Renewal

Operator picks a plan in **Subscriptions** → **Plans**, or edits an existing plan (features, limits, trial days, pricing). When a provider subscribes, the record shows up in **Subscriptions**. At renewal time, an invoice is generated (14% VAT auto-applied) and delivered. The subscription transitions through *trial* → *active* → *past_due* → *cancelled/expired*. Operator can apply discounts, extend trials, refund (full or partial), or cancel early.

2.4.3 Weekly Finance Close

Each Monday, the finance role opens **Finance**. KPIs show total revenue, commissions earned, pending payouts, and a 3-month revenue forecast with $\pm 15\%$ confidence band produced by the analytics library. Commissions are collected or waived; payouts are processed in bulk. Tax reports are exported as CSV/PDF for the accounting team.

2.4.4 Review Moderation

A review is automatically flagged if it crosses abuse thresholds, or a user reports it. It lands in **Reviews** with status *flagged*. Moderator reads the comment, checks the reporter count, and either publishes, hides, or posts an admin response. Review Analytics give a sentiment and distribution overview.

2.4.5 Incident Handling

A burst of suspicious actions on **Audit Logs** is detected by the anomaly engine and shown as a high-severity overlay. Super Admin opens **Settings** → **Security**, revokes the affected session, whitelists the legit IPs, and pushes a system-wide announcement via **Content** → **Announcements**. A postmortem is drafted via the engineering incident-response skill.

3. Tech Stack & Dependencies

The stack was chosen for three properties: **type safety** (TypeScript + Zod at runtime), **productivity** (Next.js App Router, react-hook-form), and **zero-lock-in visuals** (CSS Modules + design tokens, not a component-library monoculture). Everything below is declared in package.json at the repo root.

Package	Version	Role
next	16.2.3	App Router, file-based routing, middleware, server components.
react / react-dom	19.2.4	UI rendering, concurrent features, use() hook.
typescript	^5	Static type system (strict mode on).
zod	^4.3.6	Runtime schema validation for forms, API responses, and mocks.
react-hook-form	^7.72.1	Form state management with minimal re-renders.
@hookform/resolvers	^5.2.2	Bridges Zod schemas to react-hook-form.
recharts	^3.8.1	Area, Pie, Bar, Line, Radar charts for analytics pages.
framer-motion	^12.38.0	Micro-interactions and modal/slide-over animations.
lucide-react	^1.8.0	Icon set (Users, Settings, Shield, ...) used throughout the UI.
cmdk	^1.1.1	Command-palette primitive (Ctrl+K).
date-fns	^4.1.0	Date formatting / manipulation.
sonner	^2.0.7	Alternative toast primitive (coexists with internal useToast).

Note for engineers. The repo's AGENTS.md warns that this Next.js version has breaking changes relative to older App-Router conventions. Always consult node_modules/next/dist/docs/ before shipping new routing or middleware code. Heed deprecation notices.

4. Architecture

4.1 High-Level Architecture



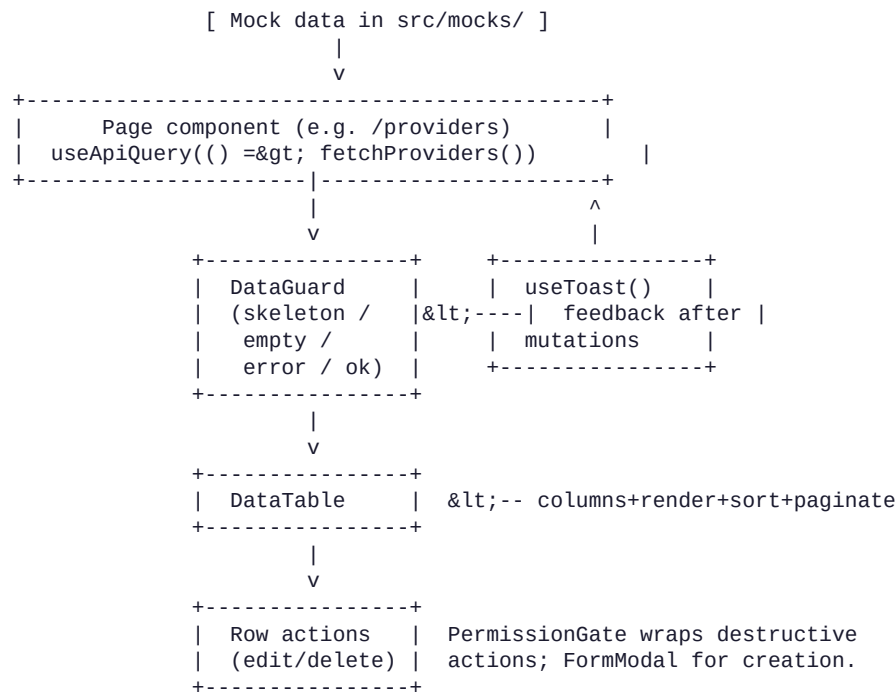
4.2 Rendering Model

All pages in `src/app/` are **client components**. The root `layout.tsx` stays server-side to install providers (Theme, Language, Auth) and mount `<AppShell>`. Pages are client components so they can use hooks, Context, and browser-only APIs (`localStorage`, `matchMedia`). Middleware runs at the edge and does the first authentication and role check before the page ever renders.

4.3 Folder Layout

```
Hagzy-SuperAdmin/
■ ■ AGENTS.md                # Agent working rules
■ ■ CLAUDE.md                # @AGENTS.md
■ ■ next.config.* / tsconfig.json
■ ■ package.json             # Next 16.2, React 19, Zod, recharts, ...
■ ■ public/                  # SVGs, favicon, (sw.js planned)
■ ■ src/
  ■ ■ app/                   # App Router
  ■   ■ layout.tsx           # Root providers + AppShell
  ■   ■ globals.css
  ■   ■ responsive.css
  ■   ■ page.tsx             # / dashboard
  ■   ■ login/               # public
  ■   ■ forgot-password/     # public
  ■   ■ providers/ (registrations, [id])
  ■   ■ users/ (wallets, [id])
  ■   ■ subscriptions/ (plans, invoices)
  ■   ■ finance/ (commissions, payouts, invoices, tax-reports)
  ■   ■ reviews/ (analytics)
  ■   ■ marketing/ (campaigns, ads/, banners, featured, promo-codes,
  ■               push-notifications)
  ■   ■ content/ (announcements, categories, pages, templates)
  ■   ■ reports/ (revenue, bookings, users, providers, geographic)
  ■   ■ support/ [id]/
  ■   ■ app-versions/
  ■   ■ system-health/
  ■   ■ audit-logs/
  ■   ■ localization/
  ■   ■ notifications/
  ■   ■ settings/ (admins, appearance, countries, localization,
  ■               payments, platform, roles, security)
  ■ ■ components/
  ■   ■ admin/               # PermissionGate, FormModal, EmptyState, StatusBadge
  ■   ■ layout/              # AppShell, Sidebar, TopBar, MobileBottomNav, Breadcrumbs
  ■   ■ tables/              # DataTable
  ■   ■ ui/                  # Button, Input, Modal, Tabs, Timeline, KPICard ...
  ■   ■ CommandPalette.tsx
  ■   ■ DataGuard.tsx
  ■   ■ ErrorBoundary.tsx
  ■   ■ OfflineBanner.tsx
  ■   ■ ui/Skeleton.tsx
  ■ ■ contexts/              # AuthContext, ThemeContext, LanguageContext,
  ■   ■                      # (NotificationsContext, SidebarContext)
  ■ ■ hooks/                 # usePermission, useTranslation, useDebounce,
  ■   ■                      # usePagination, useValidatedForm, useApiQuery,
  ■   ■                      # useUnsavedChanges, useServiceWorker,
  ■   ■                      # useAnnounce, useFocusTrap
  ■ ■ lib/                   # api.ts, permissions.ts, utils.ts, validations.ts,
  ■   ■                      # analytics.ts
  ■ ■ middleware.ts          # public routes + RBAC + cookie check
  ■ ■ mocks/                 # users, providers, subscriptions, finance, reviews,
  ■   ■                      # dashboard, notifications, experiments
  ■ ■ types/                 # admin, user, subscription, provider, review,
  ■   ■                      # finance, marketing, content, wallet, ticket,
  ■   ■                      # system, index
```

4.4 Data Flow



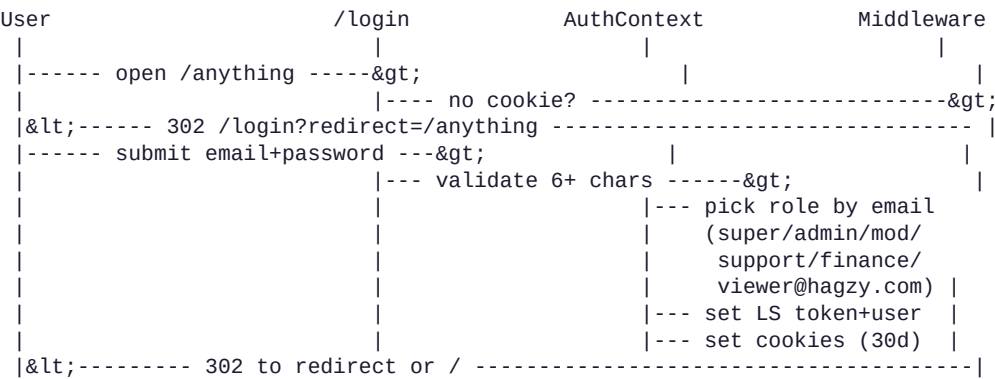
4.5 Provider Tree

```
<html lang="data-theme">
  <body>
    <ThemeProvider> # src/contexts/ThemeContext.tsx
    <LanguageProvider> # src/contexts/LanguageContext.tsx
    <AuthProvider> # src/contexts/AuthContext.tsx
    <AppShell> # src/components/layout/AppShell.tsx
    <SidebarProvider> # src/components/layout/SidebarContext.tsx
    <CommandPaletteProvider>
    <ErrorBoundary>
    <ToastProvider>
    <OfflineBanner />
    <Sidebar /> <TopBar /> <MobileBottomNav />
    {page} # src/app/**/page.tsx
    </ToastProvider>
    </ErrorBoundary>
    </CommandPaletteProvider>
    </SidebarProvider>
    </AppShell>
    </AuthProvider>
    </LanguageProvider>
    </ThemeProvider>
  </body>
</html>
```

5. Authentication & Authorization

Authentication uses two cookies (hagzy_superadmin_logged_in and hagzy_superadmin_auth) plus localStorage for hagzy_superadmin_user and hagzy_superadmin_token. Cookies allow the Next.js middleware to make a routing decision on the edge before the page renders; localStorage lets the AuthContext hydrate UI state on the client.

5.1 Login Flow



5.2 Public vs Protected Routes

```
// src/middleware.ts
const PUBLIC_ROUTES = ['/login', '/forgot-password'];

const ADMIN_ONLY_ROUTES = [
  '/settings/security',
  '/settings/roles',
  '/settings/admins',
  '/audit-logs',
];

// pseudo-code of middleware:
if (isPublic(pathname)) return next();
if (isStatic(pathname)) return next();
if (!cookies['hagzy_superadmin_logged_in'])
  return redirect('/login?redirect=' + pathname);
if (isAdminOnly(pathname) &&& role !== 'super_admin' &&& role !== 'admin')
  return redirect('/');
return next();
```

5.3 RBAC: The Permission Matrix

Each role has a hard-coded permission map in src/lib/permissions.ts. The matrix below lists every module and every action, summarizing what each role can do. Use this when deciding where to place a <PermissionGate> or when writing a new feature.

Module	super_admin	admin	moderator	support	finance	viewer
dashboard	all	CRUD+export	view	view	view	view
providers	all	CRUD+export	view	view	view	view

Module	super_admin	admin	moderator	support	finance	viewer
users	all	CRUD+export	view	view	—	view
subscriptions	all	CRUD+export	—	—	CRUD+export	view
reviews	all	CRUD+export	CRUD	view	—	view
reports	all	CRUD+export	—	—	view+export	view
finance	all	CRUD+export	—	—	CRUD+export	view
marketing	all	CRUD+export	CRUD	—	—	view
support	all	CRUD+export	create+edit	CRUD	—	view
content	all	CRUD+export	CRUD	—	—	view
settings	all	CRUD+export	—	—	—	view
app_versions	all	CRUD+export	—	—	—	view
system_health	all	CRUD+export	view	view	view	view
audit_logs	all	view+export	—	—	—	view
localization	all	CRUD+export	CRUD	—	—	view
wallets	all	CRUD+export	—	view	view+export	view
ads	all	CRUD+export	CRUD	—	—	view

Legend — CRUD = create/edit/delete/view. “all” includes impersonate. Only super_admin can impersonate a provider.

5.4 Permission Gates in Code

```
// Component usage
import PermissionGate from '@components/admin/PermissionGate';

<PermissionGate module="users" action="delete"
  fallback={<Button disabled>Delete</Button>}>>
  <Button variant="destructive" onClick={handleDelete}>Delete</Button>
</PermissionGate>

// Hook usage
import { usePermission } from '@hooks/usePermission';

const { can, canAny, canAll } = usePermission();
if (can('subscriptions', 'edit')) { /* show Edit button */ }
```

5.5 Impersonation

Super admins can “become” a provider to reproduce an issue. `startImpersonating(providerId, providerName)` stashes the session in `localStorage` and is surfaced in the `TopBar` as a red banner. `stopImpersonating()` clears it. Every impersonation action is written to the audit log and, because it is a high-risk action, is also flagged by the anomaly engine.

6. Complete Route Map

Hagzy SuperAdmin has 50+ routes. The table below lists every route with its file, the module it belongs to, and the access level enforced by middleware. For the deeper behavioral description of each page, see **Section 7 — Module Deep Dive**.

Route	File	Module	Access
/login	app/login/page.tsx	Auth	Public
/forgot-password	app/forgot-password/page.tsx	Auth	Public
/	app/page.tsx	Dashboard	Authenticated
/providers	app/providers/page.tsx	Providers	Authenticated
/providers/[id]	app/providers/[id]/page.tsx	Providers	Authenticated
/providers/registrations	app/providers/registrations/page.tsx	Providers	Authenticated
/users	app/users/page.tsx	Users	Authenticated
/users/[id]	app/users/[id]/page.tsx	Users	Authenticated
/users/wallets	app/users/wallets/page.tsx	Users/Wallets	Authenticated
/subscriptions	app/subscriptions/page.tsx	Subscriptions	Authenticated
/subscriptions/plans	app/subscriptions/plans/page.tsx	Subscriptions	Authenticated
/subscriptions/invoices	app/subscriptions/invoices/page.tsx	Subscriptions	Authenticated
/finance	app/finance/page.tsx	Finance	Authenticated
/finance/commissions	app/finance/commissions/page.tsx	Finance	Authenticated
/finance/payouts	app/finance/payouts/page.tsx	Finance	Authenticated
/finance/invoices	app/finance/invoices/page.tsx	Finance	Authenticated
/finance/tax-reports	app/finance/tax-reports/page.tsx	Finance	Authenticated
/reviews	app/reviews/page.tsx	Reviews	Authenticated
/reviews/analytics	app/reviews/analytics/page.tsx	Reviews	Authenticated
/marketing/campaigns	app/marketing/campaigns/page.tsx	Marketing	Authenticated
/marketing/ads	app/marketing/ads/page.tsx	Marketing	Authenticated
/marketing/ads/experiments	app/marketing/ads/experiments/page.tsx	Marketing	Authenticated
/marketing/banners	app/marketing/banners/page.tsx	Marketing	Authenticated
/marketing/featured	app/marketing/featured/page.tsx	Marketing	Authenticated
/marketing/promo-codes	app/marketing/promo-codes/page.tsx	Marketing	Authenticated
/marketing/push-notifications	app/marketing/push-notifications/page.tsx	Marketing	Authenticated

Route	File	Module	Access
/content/announcements	app/content/announcements/page.tsx	Content	Authenticated
/content/pages	app/content/pages/page.tsx	Content	Authenticated
/content/categories	app/content/categories/page.tsx	Content	Authenticated
/content/templates	app/content/templates/page.tsx	Content	Authenticated
/reports	app/reports/page.tsx	Reports	Authenticated
/reports/revenue	app/reports/revenue/page.tsx	Reports	Authenticated
/reports/bookings	app/reports/bookings/page.tsx	Reports	Authenticated
/reports/users	app/reports/users/page.tsx	Reports	Authenticated
/reports/providers	app/reports/providers/page.tsx	Reports	Authenticated
/reports/geographic	app/reports/geographic/page.tsx	Reports	Authenticated
/support	app/support/page.tsx	Support	Authenticated
/support/[id]	app/support/[id]/page.tsx	Support	Authenticated
/app-versions	app/app-versions/page.tsx	App Versions	Authenticated
/system-health	app/system-health/page.tsx	System Health	Authenticated
/audit-logs	app/audit-logs/page.tsx	Audit Logs	Admin only
/localization	app/localization/page.tsx	Localization	Authenticated
/notifications	app/notifications/page.tsx	Notifications	Authenticated
/settings/appearance	app/settings/appearance/page.tsx	Settings	Authenticated
/settings/admins	app/settings/admins/page.tsx	Settings	Admin only
/settings/roles	app/settings/roles/page.tsx	Settings	Admin only
/settings/security	app/settings/security/page.tsx	Settings	Admin only
/settings/payments	app/settings/payments/page.tsx	Settings	Authenticated
/settings/countries	app/settings/countries/page.tsx	Settings	Authenticated
/settings/localization	app/settings/localization/page.tsx	Settings	Authenticated
/settings/platform	app/settings/platform/page.tsx	Settings	Authenticated

7. Module Deep Dive

Each subsection describes a module as an engineer, an operator, and a stakeholder would care about it — what it does, what the data looks like, and how the operator uses it.

7.1 Dashboard (/)

The home page is a read-only “single screen of truth.” Six KPI cards at the top (providers, users, bookings, revenue, subscriptions, pending tickets) with trend arrows; a revenue AreaChart over 12 months; a PieChart of bookings by category; a subscription tier distribution; top-five providers by revenue; a live activity feed; and a churn-risk alert panel powered by `analytics.assessChurnRisk()`.

Operator tasks

- Spot revenue drop — click the chart to drill into **Reports** → **Revenue**.
- Investigate pending tickets — jump to **Support** from the KPI card.
- Triage a high-risk provider — click their name in the churn-risk panel.

7.2 Providers

/providers

DataTable of every registered provider with columns: name, email, subscription tier, total bookings, revenue, rating, and a computed **performance score** (elite, strong, average, at_risk) from `scoreProvider()`. Filter by status and category. Row actions: view, block/unblock, delete, impersonate (super admin). Export CSV.

/providers/[id]

Detail view: profile, subscription card, commission rate, performance breakdown (bookings, revenue, activity, subscription, tenure sub-scores), branches, employees, recent bookings, financial summary.

/providers/registrations

Approval queue. Columns: `business_name`, `category`, `submitted_at`, `status` (pending / approved / rejected). Approve with optional edits, reject with reason. Uploaded documents preview inline.

Performance scoring weights: bookings 30%, revenue 25%, activity / recency 20%, subscription status 15%, tenure 10%. Tiers: elite 80+, strong 60-79, average 40-59, at_risk <40.

7.3 Users & Wallets

/users

DataTable with name, email, total bookings, wallet balance, last active. Statuses: active, blocked, suspended, soft_deleted. Creation form uses `userCreateSchema` (see Section 10.4).

/users/wallets

Wallet dashboard with balance, total credits/debits, last transaction. Actions: view transactions, adjust (add/deduct), freeze/unfreeze, refund (full or partial, destination wallet or original payment method).

```
// Adjust schema – cap deductions to current balance
walletAdjustSchema(maxDeduct?: number):
  amount: number positive, <= maxDeduct when deducting
  reason: string required

// Refund schema – partial amount capped, two destinations
refundSchema(maxAmount: number):
  type: 'partial' | 'full'
  amount: number positive, <= maxAmount
  destination: 'wallet' | 'original'
  reason: string required
```

7.4 Subscriptions

/subscriptions

Active provider subscriptions. Filter by status. Actions: upgrade, apply discount, extend trial, generate invoice, cancel, refund.

/subscriptions/plans

Plan catalog. Tiers: basic, pro, enterprise. Limits configurable (max_branches, max_employees, max_services, max_bookings_per_month, storage_gb), feature toggles, trial days 0-90.

/subscriptions/invoices

Invoice editor with line items, fixed/percent discounts, auto VAT 14%. Generates a PDF preview.

7.5 Finance

Finance hub with KPIs (Apr revenue, commissions, pending/completed payouts, next-month forecast). forecastRevenue() runs a least-squares linear regression on monthly history and extends three months with a $\pm 15\%$ confidence band. Sub-pages: commissions ledger, payouts batch processor, invoices, tax reports (VAT + withholding).

Commission states

```
pending → collected    (operator collects)
pending → waived       (operator waives, e.g. goodwill)

(no direct collected → pending reversal – must create adjustment)
```

Payout states

```
pending → processing → completed
                ■→ failed (retryable)
```

7.6 Reviews

Columns: reviewer, provider, rating (1-5), comment, status (published, flagged, hidden, pending), reported count, flag reason. Actions: publish, flag, hide, add/edit admin response. Analytics sub-page: rating distribution, trend, most-reviewed providers, sentiment.

7.7 Marketing

Six sub-modules: campaigns, ads, banners, featured, promo-codes, push-notifications.

/marketing/campaigns

Campaign types: featured_listing, banner, email, push. Columns: name, type, status (draft/active/paused/completed), budget vs spent, impressions, clicks, conversions.

/marketing/ads & /marketing/ads/experiments

Rich ad targeting (cities, categories, user segments), placements (home_banner, category_banner, search_promoted, between_listings), ad types (image_banner, promotional_card, featured_provider, video_ad), priority, schedule, analytics (CTR, conversions). Experiments page runs a two-proportion z-test via `analytics.compareVariants()`: computes lift, confidence (0-100%), declares a winner at $\geq 95\%$ confidence with $\geq 1,000$ samples.

/marketing/promo-codes

Code schema allows alphanumeric + - + _ . Discount type *percentage* validates value 0-100; *fixed* validates positive money.

/marketing/push-notifications

Bilingual push composer (en/ar). Target app (user, employee, all) and segment (all, active, inactive, new, custom). Draft → scheduled → sent, with recipients and opened counts.

7.8 Content

Four sub-modules: announcements, categories, pages, templates. Every content item has English + Arabic fields. Templates expose variables such as `{{name}}` or `{{booking_id}}` that are expanded server-side before sending.

7.9 Reports

Five analytical slices: revenue (subscriptions vs commissions with forecast), bookings (trends, peak times, cancellations, AOV), users (registrations, active, churn, LTV, cohorts), providers (performance distribution, subscription status, churn heatmap), geographic (by city / country). All support date-range filters and CSV/PDF export.

7.10 Support

Ticket list with SLA deadline and SLA-breached flag. Categories: billing, technical, account, booking, complaint, feature_request, other. Priority: low, medium, high, urgent. Status: open, in_progress, waiting_on_customer, waiting_on_provider, resolved, closed. Detail page shows the message thread (customer, admin, system), attachments, and action buttons (reply, resolve/close, reassign, change priority).

7.11 App Versions

Separate entries for user_ios, user_android, employee_ios, employee_android. Fields: version, build_number, min_required_version, force_update toggle, bilingual release notes, released_at.

7.12 System Health

Real-time status board. Each service shows status (healthy / degraded / down), uptime %, avg response (ms), error rate, last checked. Threshold-based color coding.

7.13 Audit Logs

Every sensitive action is logged: timestamp, admin, role, action, entity_type, entity_id, JSON details, IP, user agent. Admin-only. The **anomaly overlay** powered by analytics.detectAnomalies() highlights:

- **Burst:** 5+ actions in 10 minutes from the same admin.
- **Off-hours:** actions between 00:00 and 05:00 local time.
- **High-risk:** destructive actions (delete, refund, impersonate).
- **Rapid reversal:** create then delete (or block then unblock) within minutes.

Severity: low / medium / high. Each flag carries a human-readable explanation so reviewers see *why* the action was suspicious.

7.14 Settings

/settings/appearance

Theme toggle (light/dark/system), palette editor, typography, border-radius.

/settings/admins (admin only)

Manage staff accounts. Role selector + password generator. Disable/enable.

/settings/roles (admin only)

Matrix editor. Toggle *view / create / edit / delete / export / impersonate* per module per role.

/settings/security (admin only)

2FA setup, active sessions list + revoke, login activity, IP allow/deny, permission policy editor.

/settings/payments

Gateway integrations (Stripe, PayPal...), API keys with show/hide, webhook URLs, fee rates.

/settings/countries

Country → city hierarchy with drag-and-drop reorder and bulk import.

/settings/platform

App name, support email/phone, terms & privacy URLs, default currency, timezone, feature flags, API rate limits, session timeout.

/settings/localization

Alternate route to the same translation manager as /localization.

8. Component Reference

All components live in `src/components/`. They are grouped by purpose: `admin/` (business-specific), `layout/` (chrome), `tables/` (DataTable), `ui/` (design-system primitives), and top-level utility components.

8.1 Admin Components

Component	Props	Role
PermissionGate	module: PermissionModule action: PermissionAction children fallback?: ReactNode	Conditionally render children if current user has the permission; otherwise render fallback (defaults to null).
StatusBadge	status: enum size? className?	Color-coded badge for entity status (active, suspended, pending_review ...).
FormModal	open, onClose, title onSubmit, submitLabel? submitVariant?: primary danger loading?, width?: sm md lg children	Modal wrapper for forms. Also exports ConfirmModal and FormField helpers.
EmptyState	icon, title, description, action?	Placeholder shown when list is empty. Optional action button.

8.2 Layout Components

Component	Props	Role
AppShell	children	Root layout. Hosts Sidebar, TopBar, MobileBottomNav, Breadcrumbs, OfflineBanner, ErrorBoundary, ToastProvider, CommandPalette. Skips chrome on <code>/login</code> and <code>/forgot-password</code> .
Sidebar	(context)	Collapsible nav. Menu grouped by module with active highlighting. Hidden on mobile.
TopBar	(context)	Logo, search, user menu, notifications bell, language + theme toggles.
MobileBottomNav	—	Mobile-only tab bar at screen bottom.
Breadcrumbs	items[]	Hierarchical trail with clickable ancestors.
SidebarContext	—	collapsed + mobileOpen state. <code>useSidebar()</code> hook.

8.3 Tables

A single `DataTable` is reused across every list page. It takes data, columns, and optional flags for sortable, paginated, selectable, `onRowClick`, and `rowKey`. Each column declares a render function for custom cells (badges, avatar + name, currency, etc.).

```
<DataTable
  data={providers}
  columns={[
    { key: 'name', label: 'Name', sortable: true,
      render: (_, p) => <AvatarWithName user={p} />, },
    { key: 'revenue', label: 'Revenue', align: 'right',
```



```
        render: v => formatCurrency(v, 'EGP') },
      { key: 'score', label: 'Performance',
        render: (_, p) => <StatusBadge status={scoreProvider(p, all).tier} /> },
    ]}
    paginated
    sortable
    rowKey="id"
    onClick={p => router.push(`/providers/${p.id}`)}
  />
```

8.4 Design-System UI Library

src/components/ui/index.tsx is a single file, ~436 lines, that exports every design-system primitive. All components use CSS modules backed by CSS custom properties for theming.

Primitive	Surface
Button	variant: primary secondary ghost destructive outline; size: sm md lg; fullWidth, iconOnly, disabled
Badge	color: success error warning info neutral primary purple amber; size: sm md
Input	label, hint, error message
Select	label, options[]
Textarea	multi-line text
Switch	toggle with optional label
Checkbox	checkbox with label
Modal	overlay with header, body, footer, escape + backdrop close
SlideOver	right-aligned slide-in panel
EmptyState	icon + title + description + optional action
Skeleton	variants: text, title, circle, card
Toast	ToastProvider + useToast(); success/error/warning/info, auto-dismiss 5s, undo action
Breadcrumbs	navigation trail
Tabs	tab switcher with icon support
Stepper	progress indicator
FAB	floating action button (multi-action)
Timeline	vertical event timeline
SearchBar	icon + optional shortcut hint
KPICard	icon + value + label + optional up/down trend
DropdownMenu	items with label, icon, onClick, destructive flag

8.5 Top-Level Utility Components

Component	Role
CommandPalette	Cmd/Ctrl+K palette with grouped actions (cmdk).
DataGuard	Wraps async pages: skeleton during load, error+retry on failure, empty state when data is [], children otherwise.
ErrorBoundary	Class component. Catches render errors; shows expandable error UI with reset.
OfflineBanner	Fixed bottom banner driven by useServiceWorker() online/offline events.
Skeleton	Loading shimmer in multiple variants.

9. Hooks Reference

Every custom hook lives in `src/hooks/`. Hooks are the preferred unit of reuse in this codebase — pages compose page-specific logic out of these hooks rather than inheriting from shared base components.

Hook	Signature	Role
<code>usePermission</code>	<code>() => { can(m, a), canAny(m, a[]), canAll(m, a[]) }</code>	RBAC checks. Powers and conditional UI.
<code>useTranslation</code>	<code>() => { t(key: string): string; lang: 'en' 'ar' }</code>	i18n lookup with fallback to key. Reactive to <code>LanguageContext</code> .
<code>useDebounce</code>	<code>(value: T, delay = 300) => T</code>	Debounced echo of a value. Typical use: search inputs.
<code>usePagination</code>	<code>(initialPage?, initialLimit?) => { page, limit, total, totalPages, setPage, setLimit, setTotal, next, prev, reset, hasNext, hasPrev }</code>	Pagination state. Sets <code>totalPages</code> when <code>setTotal</code> is called.
<code>useValidatedForm</code>	<code>(schema, defaults?, trackUnsaved?) => useForm & { getError, hasChanges, markSaved, confirmNavigation }</code>	react-hook-form + Zod + optional unsaved-changes guard.
<code>useApiQuery</code>	<code>(fetcher, deps?, { enabled?, fallbackData? }) => { data, loading, error, refetch, setData }</code>	GET-like fetching with dep-driven refetch and mount safety.
<code>useApiMutation</code>	<code>(mutator) => { mutate(vars) → T null, data, loading, error }</code>	POST/PUT/DELETE-like mutations.
<code>useUnsavedChanges</code>	<code>(enabled?) => { hasChanges, setHasChanges, confirmNavigation, markSaved }</code>	beforeunload guard and programmatic navigation prompts.
<code>useServiceWorker</code>	<code>() => { isOffline, isReady }</code>	Registers SW (production only), 60-second update check, online/offline events.
<code>useAnnounce</code>	<code>(politeness?) => (message: string) => void</code>	aria-live announcements for screen readers.
<code>useFocusTrap</code>	<code>({ isActive?, onEscape?, autoFocus? }) => Ref</code>	Traps focus inside modals; Escape triggers <code>onEscape</code> .

9.1 Hook Usage Examples

useValidatedForm

```
import { useValidatedForm } from '@/hooks/useValidatedForm';
import { providerCreateSchema } from '@/lib/validations';

const form = useValidatedForm(providerCreateSchema, {
  business_name: '',
  name: '',
  email: '',
  phone: '',
  business_category: 'salon',
  country: 'Egypt',
  city: '',
  commission_rate: 15,
}, /* trackUnsavedChanges */ true);

<form onSubmit={form.handleSubmit(onCreate)}>
  <Input label="Business name" {...form.register('business_name')}
    error={form.getError('business_name')} />
  { /* ... */ }
</form>
```

useApiQuery + DataGuard

```
const { data, loading, error, refetch } = useApiQuery(
  () => api.get<User[]>('/users'),
  [filters],
);

<DataGuard loading={loading} error={error} data={data}
  emptyIcon={<Users />}
  emptyTitle="No users yet"
  emptyDescription="Invite your first user to get started."
  onRetry={refetch}>
  <DataTable data={data!} columns={columns} paginated />
</DataGuard>
```

useFocusTrap

```
function ConfirmModal({ open, onClose }: Props) {
  const ref = useFocusTrap({ isActive: open, onEscape: onClose, autoFocus: true });
  if (!open) return null;
  return <div ref={ref} role="dialog" aria-modal> ... </div>;
}
```

10. Context Providers

Three providers are installed in `layout.tsx` before the `AppShell`. Two more (`SidebarContext`, `NotificationsContext`) are installed lower in the tree.

10.1 AuthContext

Manages the logged-in `SuperAdminUser`, loading state, and the optional impersonation session. All authentication APIs are mocked but their signatures match a real backend.

```
const {
  user,                // SuperAdminUser | null
  loading,             // boolean
  impersonating,       // { providerId, providerName, startedAt } | null
  login,               // (email, password) => Promise<{ success, user?, error? }>;
  logout,              // () => void
  forgotPassword,      // (email) => Promise<{ success, error? }>;
  verifyOtpCode,       // (email, otp) => Promise<{ success, error? }>;
  resetPassword,       // (email, otp, newPassword) => Promise<{ success, error? }>;
  startImpersonating,
  stopImpersonating,
} = useAuth();
```

Storage: cookies (`hagzy_superadmin_logged_in`, `hagzy_superadmin_auth`, 30-day expiry) + `localStorage` (`hagzy_superadmin_user`, `hagzy_superadmin_token`). **Mock login:** accepts any email and a password of 6+ characters. Emails like `admin@hagzy.com`, `finance@hagzy.com`, etc. resolve to the matching role; other emails default to `super_admin`. **Mock OTP:** 123456.

10.2 ThemeContext

```
const { theme, resolvedTheme, setTheme, toggleTheme } = useTheme();
// theme: 'light' | 'dark' | 'system'
// resolvedTheme: 'light' | 'dark' (system → detected via matchMedia)
// Persists to localStorage.hagzy_theme
// Sets <html data-theme="..."> for CSS to react.
```

10.3 LanguageContext

```
const { language, setLanguage, toggleLanguage, isRTL } = useLanguage();
// language: 'en' | 'ar'
// isRTL: true when language === 'ar'
// Sets <html lang="..." dir="ltr|rtl">
// Persists to localStorage.hagzy_settings.language
```

10.4 NotificationsContext

```
const { notifications, unreadCount, markAsRead, markAllAsRead, dismiss }
  = useNotifications();
// AdminNotification[] seeded from src/mocks/notifications.ts
```

10.5 SidebarContext

```
const { collapsed, toggleSidebar, mobileOpen, setMobileOpen } = useSidebar();
```

11. Type Reference

Every domain entity has a dedicated file in `src/types/`. The summary below lists the most important unions and interfaces; see the actual files for every field.

11.1 Admin & Permissions (admin.ts)

```
type SuperAdminRole =
  | 'super_admin' | 'admin' | 'moderator'
  | 'support'      | 'finance' | 'viewer';

type PermissionModule =
  | 'dashboard' | 'providers' | 'users' | 'subscriptions'
  | 'reviews'   | 'reports'   | 'finance' | 'marketing'
  | 'support'   | 'content'   | 'settings' | 'app_versions'
  | 'system_health' | 'audit_logs' | 'localization'
  | 'wallets'    | 'ads';

type PermissionAction =
  | 'view' | 'create' | 'edit' | 'delete' | 'export' | 'impersonate';

interface Permission {
  module: PermissionModule;
  actions: PermissionAction[];
}

interface SuperAdminUser {
  id: string; uuid: string;
  name: string; email: string; phone?: string; avatar_url?: string;
  role: SuperAdminRole;
  permissions: Permission[];
  active: boolean;
  last_login_at: string;
  created_at: string; updated_at: string;
}
```

11.2 Providers (provider.ts)

```

type ProviderStatus =
  | 'active' | 'suspended' | 'blocked' | 'soft_deleted'
  | 'pending_review' | 'rejected' | 'deactivated';

type BusinessCategory =
  'salon' | 'barber' | 'clinic' | 'spa' | 'nails' | 'other';

interface Provider {
  id, uuid, name, email, phone,
  business_name, business_category: BusinessCategory,
  status: ProviderStatus,
  subscription_plan_id?: string,
  subscription_status: 'active' | 'trial' | 'past_due' | 'cancelled' | 'expired',
  country, city,
  branches_count: number, employees_count: number,
  total_bookings: number, total_revenue: number,
  commission_rate: number,
  registered_at, verified_at?, last_active_at,
  created_at, updated_at, deleted_at?
}

interface ProviderRegistration {
  id, provider_name, business_name, business_category,
  email, phone,
  documents: string[],
  status: 'pending' | 'approved' | 'rejected',
  submitted_at, reviewed_by?, reviewed_at?, rejection_reason?
}

```

11.3 Users & Wallets (user.ts, wallet.ts)

```

type UserStatus = 'active' | 'blocked' | 'suspended' | 'soft_deleted';

interface PlatformUser {
  id, uuid, name, email, phone, avatar_url?,
  status: UserStatus,
  gender?, date_of_birth?,
  country, city, preferred_language?,
  total_bookings, total_spent, wallet_balance,
  last_booking_at?, last_active_at, registered_at,
  created_at, updated_at, deleted_at?
}

type WalletStatus = 'active' | 'frozen';
type WalletAction = 'add' | 'deduct' | 'freeze' | 'unfreeze' | 'refund';

interface Wallet {
  id, user_id, user_name, user_email,
  balance, currency, status: WalletStatus,
  total_credits, total_debits,
  last_transaction_at?, created_at, updated_at
}

interface WalletTransaction {
  id, wallet_id,
  type: 'credit' | 'debit',
  action: WalletAction | 'payment' | 'booking_refund' | 'top_up',
  amount, balance_after, description,
  reference_id?, performed_by?, created_at
}

```

11.4 Subscriptions (subscription.ts)

```
type PlanTier      = 'basic' | 'pro' | 'enterprise';
type BillingCycle   = 'monthly' | 'quarterly' | 'yearly';

interface PlanLimits {
  max_branches, max_employees, max_services,
  max_bookings_per_month, storage_gb
}

interface PlanFeature {
  key: string; label: string; label_ar: string; included: boolean;
}

interface SubscriptionPlan {
  id, name, name_ar, tier: PlanTier,
  price_monthly, price_yearly, currency,
  features: PlanFeature[], limits: PlanLimits,
  active: boolean, trial_days: number,
  providers_count: number,
  created_at, updated_at
}

interface ProviderSubscription {
  id, provider_id, provider_name,
  plan_id, plan_name, plan_tier: PlanTier,
  billing_cycle: BillingCycle,
  status: 'active' | 'trial' | 'past_due' | 'cancelled' | 'expired',
  current_period_start, current_period_end, trial_end?,
  amount, currency, payment_method
}
```


11.5 Finance & Marketing & Content

```
// finance.ts
interface CommissionRecord {
  id, provider_id, provider_name, booking_id,
  booking_amount, commission_rate, commission_amount,
  status: 'pending' | 'collected' | 'waived',
  created_at
}

interface PayoutRecord {
  id, provider_id, provider_name, amount, currency,
  method: 'bank_transfer' | 'wallet' | 'check',
  status: 'pending' | 'processing' | 'completed' | 'failed',
  period_start, period_end, created_at, completed_at?
}

// marketing.ts
interface Ad {
  id, title, title_ar, description, description_ar,
  image_url, target_url,
  placement: 'home_banner' | 'category_banner'
    | 'search_promoted' | 'between_listings',
  ad_type: 'image_banner' | 'promotional_card'
    | 'featured_provider' | 'video_ad',
  targeting: { cities: string[]; categories: string[]; user_segments: string[] },
  schedule: { start_date: string; end_date: string },
  priority, status,
  analytics: { impressions, clicks, ctr, conversions },
  created_at, updated_at
}

// content.ts
interface ContentPage {
  id, slug, title, title_ar, content, content_ar,
  type: 'terms' | 'privacy' | 'faq' | 'about' | 'custom',
  published, last_updated_by, created_at, updated_at
}

interface EmailTemplate {
  id, name, slug, subject, subject_ar, body_html, body_html_ar,
  variables: string[],
  type: 'email' | 'sms',
  active, created_at, updated_at
}
```

11.6 System (system.ts) & Support (ticket.ts)

```
interface AppVersion {
  id,
  app: 'user_ios' | 'user_android'
    | 'employee_ios' | 'employee_android',
  version, build_number, min_required_version,
  force_update, release_notes, release_notes_ar,
  released_at, created_at
}

interface SystemHealthMetric {
  service,
  status: 'healthy' | 'degraded' | 'down',
  uptime_percent, avg_response_ms, error_rate, last_checked_at
}

interface AuditLog {
  id, admin_id, admin_name, admin_role,
  action, entity_type, entity_id?, details,
  ip_address, user_agent, created_at
}

type TicketPriority = 'low' | 'medium' | 'high' | 'urgent';
type TicketStatus =
  'open' | 'in_progress' | 'waiting_on_customer'
  | 'waiting_on_provider' | 'resolved' | 'closed';

interface SupportTicket {
  id, subject, description,
  category, priority, status,
  submitted_by: { type, id, name, email },
  assigned_to?, assigned_to_name?,
  sla_deadline?, sla_breached,
  messages: TicketMessage[], tags: string[],
  created_at, updated_at, resolved_at?
}
```

12. Library Reference (src/lib)

12.1 api.ts

Singleton ApiClient exporting api. Base URL defaults to `https://waqty.alemdayaz.shop/public`. Reads `hagzy_token` from `localStorage` and attaches `Authorization: Bearer ...`; also forwards `Accept-Language`. Methods: `get`, `post`, `put`, `patch`, `delete`, `postFormData`. Response type: `{ success, message, data? }`.

12.2 permissions.ts

Source of truth for role–module–action relationships. Exports:

- `ALL_MODULES` — the 18 permission modules.
- `ROLE_PERMISSIONS` — the six-role matrix (see Section 5.3).
- `getPermissionsForRole(role)` — returns an array of `Permission` for a given role.
- `hasPermission(permissions[], module, action)` — predicate used by `usePermission()` and `<PermissionGate>`.

12.3 utils.ts

```
exportToCSV<T>(data: T[], filename: string, columns?): void
// Auto-escapes commas and quotes, auto-suffixes filename with date.

copyToClipboard(text: string): Promise<boolean>
// Uses Clipboard API with execCommand fallback.

formatCurrency(amount: number, currency = 'EGP'): string
// Locale-aware, adds thousands separators and currency prefix.

formatDate(dateStr: string): string
// ISO → locale date string; returns '-' if null/invalid.
```

12.4 validations.ts

Zod schemas for all forms. Base schemas compose higher-level entity schemas. Every mutating form in the app passes through one of these.

```
// Base
emailSchema, egyptPhoneSchema    // +20 / 020 / 0 + 1[0125] + 8 digits
passwordSchema                   // min 8, uppercase, number, special
requiredString(label, min = 1), optionalUrl
percentSchema, commissionRateSchema
positiveMoneySchema, positiveIntSchema

// Entities
providerCreateSchema             // business_name, name, email, phone, category,
                                // country, city, commission_rate (default 15)
userCreateSchema                 // name, email, phone, city, country='Egypt',
                                // gender?, date_of_birth?, language='en'
adminCreateSchema                // name, email, role, password
planCreateSchema                 // name, name_ar, tier, price_monthly,
                                // price_yearly, trial_days 0..90 (default 14)
promoCodeSchema                 // code (alnum + - + _), discount_type,
                                // discount_value (validated per type),
                                // max_uses?, expires_at?
```

```
adCreateSchema      // title, description, ad_type, placement,
                    // start_date, end_date, budget;
                    // refines end_date >= start_date
announcementSchema  // title, body, target (all/providers/users/
                    // admins), scheduled_at?
countryCreateSchema // name, ISO code (2), currency (3), phone_code,
                    // active (true)
walletAdjustSchema(maxDeduct?)
refundSchema(maxAmount)
```

12.5 analytics.ts

Mock implementations of analytics routines. All functions are pure and deterministic given the same input, so they are safe to memoize.

Function	What it does
linearRegression(values[])	Least-squares fit. Returns { slope, intercept, r2 }.
forecastRevenue(history, periods=3)	Linear regression forecast with $\pm 15\%$ confidence band, extending `periods` months ahead.
detectAnomalies(logs[])	Rules: burst (≥ 5 actions / 10 min), off-hours (00:00-05:00), high-risk actions (delete/refund/impersonate), rapid reversal. Severity: low / medium / high.
scoreProvider(provider, all)	Composite 0-100 score weighted bookings 30%, revenue 25%, activity 20%, subscription 15%, tenure 10%. Tiers: elite / strong / average / at_risk.
assessChurnRisk(provider, all)	Signals: inactive > 30 days, subscription past_due/expired, engagement < 50 bookings, performance < 40. Levels: none / watch / high.
compareVariants(A, B)	Two-proportion z-test. Returns CTR, conversion, lift %, confidence 0-100, winner at $\geq 95\%$ & $\geq 1,000$ samples.

13. Styling, Theming & i18n

13.1 Styling

The codebase intentionally avoids a CSS framework. Styling is **CSS Modules** plus **CSS custom properties** defined in `globals.css`. This keeps components decoupled from any framework upgrade path and makes dark mode a one-attribute flip.

```
:root {
  /* Colors */
  --color-primary-500: #2e75b6;
  --color-success: #16a34a;
  --color-warning: #d97706;
  --color-error: #dc2626;
  --color-info: #0ea5e9;

  /* Spacing: baseline multiples */
  --space-1: 4px; --space-2: 8px; --space-3: 12px;
  --space-4: 16px; --space-5: 20px; --space-6: 24px;

  /* Typography */
  --text-xs: 12px; --text-sm: 14px; --text-base: 16px;
  --text-lg: 18px; --text-xl: 22px;
  --font-regular: 400; --font-medium: 500; --font-semibold: 600;

  /* Radius + shadow */
  --radius-sm: 4px; --radius-md: 8px; --radius-lg: 12px; --radius-full: 9999px;
  --shadow-sm: ...; --shadow-md: ...; --shadow-lg: ...; --shadow-xl: ...;

  /* Theme tokens */
  --bg-primary: #ffffff; --bg-secondary: #f3f4f6;
  --text-primary: #111827; --text-secondary: #374151;
}

[data-theme='dark'] {
  --bg-primary: #0b1020; --bg-secondary: #111827;
  --text-primary: #f9fafb; --text-secondary: #d1d5db;
  /* ... */
}
```

13.2 Theming

`ThemeProvider` sets `data-theme` on `<html>` every time the resolved theme changes, and subscribes to `prefers-color-scheme` so the OS change applies immediately when the user chose *system*. The `TopBar` Sun/Moon button calls `toggleTheme()`.

13.3 Internationalization

`LanguageProvider` sets `lang` and `dir` on `<html>`. When `dir="rtl"`, the entire UI mirrors naturally because all layout uses logical CSS properties (`margin-inline`, `padding-inline`) rather than `left/right`. Translation keys live in `src/i18n/translations/` and follow dotted paths (`users.title`, `finance.commissions`, ...). Missing keys log a dev warning and fall back to the key itself — visible during development so nothing ships untranslated.

13.4 Responsive Breakpoints

Breakpoint	Width	Behavior
Desktop	> 1024 px	Sidebar + TopBar + MobileBottomNav hidden. Dense data tables.
Tablet	768-1024 px	Sidebar auto-collapses. Some tables go to card view.
Mobile	< 768 px	Sidebar hidden behind a drawer. MobileBottomNav shown. KPI cards stack.

13.5 Accessibility Summary

- Semantic HTML (<button>, <nav>, role="dialog").
- useFocusTrap on modals + aria-modal="true".
- useAnnounce for screen-reader announcements after async ops.
- Keyboard: Cmd/Ctrl+K palette, Tab inside modals, Escape closes.
- Color contrast meets WCAG AA in both themes (verify new components with the design:accessibility-review skill).
- aria-label on icon-only buttons.

14. Mock Data Layer

Everything under `src/mocks/` is seeded test data. Because the frontend already type-checks against the `src/types/` shapes, swapping mocks for real API responses is a one-line change in `useApiQuery` callers.

File	Exports	Shape / Coverage
<code>users.ts</code>	<code>mockUsers</code> , <code>mockWallets</code> , <code>mockWalletTransactions</code>	10 platform users across active/blocked/suspended/soft_deleted; wallets derived; 7 transactions.
<code>providers.ts</code>	<code>mockProviders</code> , <code>mockProviderRegistrations</code>	15+ providers across categories with revenue and booking counts; pending/approved/rejected registrations.
<code>subscriptions.ts</code>	<code>mockSubscriptions</code> , <code>mockPlans</code>	~20 active/trial/past_due subs; basic/pro/enterprise plans.
<code>finance.ts</code>	<code>mockCommissions</code> , <code>mockPayouts</code> , <code>monthlyRevenueData</code>	Commission ledger, payout batches, 12 months of revenue.
<code>reviews.ts</code>	<code>mockReviews</code>	Published/flagged/hidden/pending reviews.
<code>dashboard.ts</code>	<code>mockKPIs</code> , <code>mockRevenueData</code> , <code>mockCategoryBreakdown</code> , <code>mockSubscriptionBreakdown</code> , <code>mockTopProviders</code> , <code>mockRecentActivity</code>	All data for the home page.
<code>notifications.ts</code>	<code>mockNotifications</code>	Seed for <code>NotificationsContext</code> .
<code>experiments.ts</code>	ads A/B data	Variants for <code>/marketing/ads/experiments</code> .

15. End-User Guide

This chapter is for operators — admins, moderators, support, finance, and viewers using the dashboard day-to-day. Every section answers the same two questions: *how do I do it?* and *what can go wrong?*

15.1 Signing In

Go to `/login`. Enter your email (e.g. `support@hagzy.com`) and password. On success you are redirected to the page you tried to open, or to `/`. Password reset: click *Forgot password?*, enter your email, type the OTP (mock: 123456), set a new password (min 8 chars, one uppercase, one number, one special character).

15.2 Using the Command Palette

Press **Ctrl+K** (**⌘+K** on Mac) anywhere in the app. Type to fuzzy-search actions (“approve registration”, “new promo code”) and navigation shortcuts. Escape closes the palette.

15.3 Filters, Search, and Sort

Every list page exposes a search bar (debounced by 300 ms) and status filters. Click a column header to sort. Pagination defaults to 20 rows per page; change via the limit selector.

15.4 Exporting Data

Most list pages expose an *Export CSV* button (gated by the export permission for the module). The filename includes today's date and contains only the rows currently filtered.

15.5 Approving a Provider

- Open **Providers** → **Registrations**.
- Review uploaded documents and contact details.
- Click *Approve* (optionally adjust name, category, commission).
- If data is insufficient, click *Reject* and provide a reason.
- Approved providers appear in **Providers**; rejected ones retain an audit entry but do not appear in the main provider list.

15.6 Adjusting a User's Wallet

- Open **Users** → **Wallets**.
- Click *Adjust* on the target wallet.
- Choose *Add* or *Deduct*. Deductions are capped at the current balance automatically.
- Enter a reason (required). Save.
- The transaction appears in the wallet's ledger and in the audit log.

15.7 Processing Payouts

- Open **Finance** → **Payouts**. Filter by *pending*.
- Select one or many (checkbox in the first column). Bulk-process.
- Payouts move *pending* → *processing* → *completed*; failed ones can be retried.
- Export a statement to CSV for the accounting team.

15.8 Moderating a Review

- Open **Reviews**. Filter by *flagged* to see the queue.
- Read the comment and the flag reason.
- Pick one: *Publish*, *Hide*, or *Add admin response*.
- Track patterns in **Reviews** → **Analytics**.

15.9 Sending a Push Notification

- Open **Marketing** → **Push Notifications**.
- *New*. Compose title and body in English and Arabic.
- Target app (user / employee / all) and segment (all / active / inactive / new / custom).
- Schedule or send now. *Recipients* and *Opened* counts populate after delivery.

15.10 Responding to a Support Ticket

- Open **Support**. Sort by *priority*, filter by *open*.
- Click a ticket. Read the thread top-to-bottom.
- Reply in the composer; optionally reassign or change priority.
- Resolve when done. Closed tickets can be reopened from the detail page.
- SLA breaches are highlighted with a red badge; escalate via **Audit Logs**.

15.11 Releasing a Mobile App Version

- Open **App Versions**.
- Create an entry for the platform you are releasing (user iOS, ...).
- Enter version, build, release notes (en/ar).
- If this release is required, set *force_update* and pick the *min_required_version*.
- Save. Mobile clients see the new version on next launch.

16. Developer Onboarding

16.1 Running Locally

```
# Clone
git clone <repo-url> Hagzy-SuperAdmin && cd Hagzy-SuperAdmin

# Install (Node 20+; Next 16 expects a modern Node)
npm install

# Dev server on port 3001
npm run dev      # http://localhost:3001

# Production build + start
npm run build
npm run start
```

16.2 Project Conventions

- **Client components by default.** Every page under `src/app/` starts with 'use client'. Keep `layout.tsx` server-side.
- **Zod schemas in `src/lib/validations.ts`.** New forms add or compose a schema there; they never validate inline.
- **API calls go through `src/lib/api.ts`.** Never call `fetch` directly from a page.
- **No Redux / Zustand.** Contexts plus hooks. If you reach for a store, first try to hoist state to the nearest common ancestor.
- **Styling is CSS Modules.** Co-locate a `Foo.module.css` next to `Foo.tsx`. Use CSS custom properties from `globals.css`, not hard-coded hex.
- **Icons are lucide-react.** Don't add another icon pack.
- **Types are authoritative.** When in doubt, type the entity first in `src/types/<domain>.ts`.
- **Mocks mirror real shapes.** Edit a mock only if you also change the corresponding type — never the other way around.
- **Permissions wrap mutating actions.** Use `<PermissionGate>` or `usePermission().can(...)`.
- **Translate visible strings.** New user-facing copy must have keys in `src/i18n/translations/`.
- **Respect `AGENTS.md`.** This Next.js version has breaking changes; consult the local docs before writing routing or middleware code.

16.3 Adding a New Page

```
// src/app/<module>/<feature>/page.tsx
'use client';
import DataGuard from '@components/DataGuard';
import DataTable from '@components/tables/DataTable';
import PermissionGate from '@components/admin/PermissionGate';
import { useApiQuery } from '@hooks/useApiQuery';
import { api } from '@lib/api';

export default function FeaturePage() {
  const { data, loading, error, refetch } =
    useApiQuery(() => api.get('/endpoint'), []);

  return (
    <section>
      <h1>Feature</h1>
      <DataGuard loading={loading} error={error} data={data}
        emptyTitle="Nothing here" onRetry={refetch}>
        <DataTable data={data!} columns={[]} paginated sortable />
      </DataGuard>

      <PermissionGate module="feature_module" action="create">
        <NewItemModal onCreated={refetch} />
      </PermissionGate>
    </section>
  );
}
```

16.4 Adding a New Module Permission

- Add the module literal to PermissionModule in src/types/admin.ts.
- Add the module to ALL_MODULES in src/lib/permissions.ts.
- Decide each role's access in ROLE_PERMISSIONS.
- If the module requires admin-only access, add its URL prefix to ADMIN_ONLY_ROUTES in src/middleware.ts.
- Wrap UI entry points with <PermissionGate>.

16.5 Adding a New Zod Schema

```
// src/lib/validations.ts
import { z } from 'zod';

export const widgetCreateSchema = z.object({
  name: requiredString('name'),
  count: positiveIntSchema,
  active: z.boolean().default(true),
});

export type WidgetCreateInput = z.infer<typeof widgetCreateSchema>;
```

16.6 Git Workflow (Suggested)

- Branch from main: feat/<module>-<short-description>.
- Small PRs. One logical change per PR.

- Include screenshots for UI changes.
- Run `npm run build` locally — Next 16 catches a lot of issues at build time.
- Request review from a module owner (see CODEOWNERS).

17. Testing Strategy

No test runner is committed yet. When introducing tests, the following layering matches the shape of the codebase:

Layer	Coverage target
Unit (Vitest)	Pure functions in src/lib/ (analytics, utils, permissions). Zod schemas via safeParse.
Hook tests (Vitest + RTL)	Custom hooks, especially usePermission, useValidatedForm, usePagination, useApiQuery mount/unmount safety.
Component (RTL)	DataTable, FormModal, PermissionGate (positive/negative paths), DataGuard states.
Page (Playwright)	Login redirect, admin-only route guarding, CRUD flows for providers/users/subscriptions/promo-codes.
Accessibility (axe-core)	Run on all modal surfaces, login, dashboard, data tables.
Visual (Chromatic-style)	Storybook or Ladle for UI primitives; snapshot on push.

18. Deployment

The app is a standard Next.js 16 project and can be deployed to any platform that supports `npm run build` + `npm run start` or outputs a static build.

18.1 Recommended: Vercel

- Connect the repo. Framework preset: Next.js.
- Environment variables (see Section 18.3).
- Preview deployments on every PR; production on main.
- Set a production branch protection requiring a green build.

18.2 Self-hosted

- Node 20+.
- `npm ci && npm run build && npm run start`.
- Run behind a reverse proxy (nginx / Cloudflare) with HTTPS.
- Persist `hagzy_superadmin_*` cookies as Secure + SameSite=Lax.

18.3 Environment Variables

Variable	Example	Purpose
NEXT_PUBLIC_API_BASE_URL	https://api.hagzy.com	Base URL for ApiClient. Falls back to the hard-coded default.
NEXT_PUBLIC_ENV	production / staging / dev	Used for conditional telemetry and service-worker registration.
SESSION_COOKIE_DOMAIN	.hagzy.com	Domain for auth cookies (SSO-ready setups).

18.4 Pre-release Checklist

- **Tests:** all green on CI.
- **Permissions:** matrix still reflects product intent.
- **Feature flags:** on for staging, off for production (or vice versa).
- **i18n:** no missing-key warnings in the build log.
- **Accessibility:** axe scan on changed pages shows no new violations.
- **Rollback plan:** previous build tagged; one-command revert.

19. Analytics Heuristics (Reference)

The `src/lib/analytics.ts` module concentrates all heuristics in one place so product owners can tune them without touching pages.

19.1 Provider Performance Score

```
score(provider) =
  0.30 * booking_percentile(provider)
+ 0.25 * revenue_percentile(provider)
+ 0.20 * activity_recency(provider)      // 1 - days_since_active/90
+ 0.15 * subscription_score(provider)   // active=1, trial=.6, past_due=.3, ...
+ 0.10 * tenure_months(provider) / 24

tier =
  elite      if score >= 80
  strong     if 60 <= score < 80
  average    if 40 <= score < 60
  at_risk    if score < 40
```

19.2 Churn Risk

```
signals = [
  inactive: days_since_active > 30,          // +40
  past_due: subscription_status in ['past_due', // +30
                                     'expired'],
  low_engagement: total_bookings < 50,        // +15
  weak_performance: performance_score < 40,   // +15
]
risk = sum(signals)
level =
  high  if risk >= 60
  watch if 25 <= risk < 60
  none  if risk < 25
```

19.3 Revenue Forecast

```
forecastRevenue(history, periods = 3):
  fit = linearRegression(history.map(p => p.revenue))
  for i in 1..periods:
    predicted = fit.intercept + fit.slope * (history.length + i)
    lower_band = predicted * (1 - 0.15)
    upper_band = predicted * (1 + 0.15)
    emit { month, predicted, lower_band, upper_band, r2: fit.r2 }
```

19.4 A/B Test

```
compareVariants(A, B):
  p_pool = (A.clicks + B.clicks) / (A.impressions + B.impressions)
  se = sqrt(p_pool*(1-p_pool) * (1/A.impressions + 1/B.impressions))
  z = (B.ctr - A.ctr) / se
  confidence = two_tailed_normal(z) // 0..1
  lift = (B.ctr - A.ctr) / A.ctr // %

winner =
  B if confidence >= 0.95 and lift > 0 and min_samples >= 1000
  A if confidence >= 0.95 and lift < 0 and min_samples >= 1000
  tie if confidence < 0.95
  inconclusive if min_samples < 1000
```

19.5 Anomaly Detection

```
rules = {  
  burst:          5+ actions by same admin within 10 min -&gt; severity medium  
  off_hours:      action timestamp in [00:00, 05:00) -&gt; severity low  
  high_risk:      action in {delete, refund, impersonate} -&gt; severity medium  
  rapid_reversal: create+delete (or block+unblock) on same entity  
                  within 5 min -&gt; severity high  
}
```


20. Glossary

Term	Definition
App Router	Next.js 13+ file-based routing under <code>src/app/</code> .
Audit Log	Tamper-evident record of admin actions for compliance and forensics.
Churn Risk	Composite score predicting a provider's likelihood of leaving.
Commission	Platform's share of a booking's value, paid by the provider.
Command Palette	Keyboard-driven action launcher (Cmd/Ctrl+K).
CSS Modules	Locally-scoped CSS files (*.module.css).
Force Update	Flag on an AppVersion requiring users to upgrade before using the app.
Impersonation	Super-admin-only action to sign in as a specific provider for debugging.
KPI	Key Performance Indicator — the numbers shown on the dashboard cards.
Middleware	Edge function that runs before every Next.js page render.
Payout	Money owed by the platform to a provider for bookings.
Permission Gate	Component that renders its children only if the user has the required permission.
RBAC	Role-Based Access Control. Six roles × 18 modules × 6 actions.
Refund	Money returned to a user's wallet or original payment method.
Service Worker	Browser-installed script that enables offline and background sync.
SLA	Service Level Agreement — commitment to resolve a ticket within a deadline.
VAT	Value-Added Tax — 14% auto-applied on subscription invoices.
Zod	TypeScript-first schema validation library.

21. Appendix A — File Index

Quick lookup of every important file with a one-line purpose.

File	Purpose
src/middleware.ts	Edge middleware: public routes, cookie check, admin-only route guard.
src/app/layout.tsx	Root layout: installs Theme, Language, Auth, AppShell.
src/app/globals.css	Design tokens + base styles.
src/app/responsive.css	Breakpoint media queries.
src/app/page.tsx	Dashboard home.
src/components/layout/AppShell.tsx	Chrome: sidebar, top bar, bottom nav, breadcrumbs, error boundary, toast.
src/components/layout/Sidebar.tsx	Collapsible navigation.
src/components/layout/TopBar.tsx	Header bar with user menu, search, theme/language/notification toggles.
src/components/layout/MobileBottomNav.tsx	Mobile tab bar.
src/components/layout/Breadcrumbs.tsx	Breadcrumb trail.
src/components/layout/SidebarContext.tsx	collapsed + mobileOpen state.
src/components/tables/DataTable.tsx	Reusable data table with sort, paginate, select, render.
src/components/ui/index.tsx	Design-system primitives (Button, Input, Modal, Tabs, ...).
src/components/ui/Skeleton.tsx	Loading shimmer in several variants.
src/components/admin/PermissionGate.tsx	Conditional render based on permission.
src/components/admin/StatusBadge.tsx	Color-coded status pill.
src/components/admin/FormModal.tsx	Modal wrapper for forms + ConfirmModal + FormField.
src/components/admin/EmptyState.tsx	Empty-state placeholder.
src/components/CommandPalette.tsx	Global Cmd/Ctrl+K palette.
src/components/DataGuard.tsx	Loading/error/empty wrapper.
src/components/ErrorBoundary.tsx	Class component for render error isolation.
src/components/OfflineBanner.tsx	Offline indicator (uses useServiceWorker).
src/contexts/AuthContext.tsx	Auth state + mock login, logout, OTP, impersonation.
src/contexts/ThemeContext.tsx	Theme state (light/dark/system).
src/contexts/LanguageContext.tsx	Language state (en/ar) + isRTL.
src/hooks/usePermission.ts	RBAC checks.
src/hooks/useTranslation.ts	i18n lookup.

File	Purpose
src/hooks/useDebounce.ts	Value debounce.
src/hooks/usePagination.ts	Pagination state manager.
src/hooks/useValidatedForm.ts	react-hook-form + Zod + unsaved-changes.
src/hooks/useApiQuery.ts	Fetch + mutation hooks.
src/hooks/useUnsavedChanges.ts	beforeunload guard.
src/hooks/useServiceWorker.ts	SW registration + online/offline events.
src/hooks/useAnnounce.ts	aria-live announcements.
src/hooks/useFocusTrap.ts	Focus trap for modals.
src/lib/api.ts	ApiClient singleton (Bearer auth, JSON + multipart).
src/lib/permissions.ts	ALL_MODULES, ROLE_PERMISSIONS, hasPermission().
src/lib/utils.ts	exportToCSV, copyToClipboard, formatCurrency, formatDate.
src/lib/validations.ts	Zod schemas for every form.
src/lib/analytics.ts	Provider score, churn risk, forecast, A/B test, anomalies.
src/mocks/*.ts	Seeded data for users, providers, subscriptions, finance, reviews, dashboard, notifications, experiments.
src/types/*.ts	Entity types for admin, user, provider, subscription, review, finance, marketing, content, wallet, ticket, system.

22. Appendix B — Future Work

The code has been intentionally staged to accept the following upgrades without a rewrite.

Upgrade	Why it is cheap to do
Swap mocks for real API	Replace mock imports with useApiQuery calls against the live backend. Response shapes are already typed.
Realtime updates	Introduce SSE or WebSockets for support tickets, audit logs, and system-health metrics.
i18n expansion	Add languages beyond en/ar. Language union in LanguageContext expands; translations folder grows.
Role editor persistence	Persist ROLE_PERMISSIONS overrides to the backend so non-engineers can tune roles from /settings/roles.
Offline writes	Service worker already registers; extend with background-sync queue for audit-log-safe retry of failed mutations.
Theming extensions	High-contrast theme for accessibility; per-tenant brand colors.
Feature flags	Server-driven flag provider wired into /settings/platform.
E2E harness	Playwright project with fixtures for each role; critical-path suite.
Analytics sink	Product analytics (PostHog / Amplitude) wired into TopBar and DataTable row-click.

End of document. For corrections, submit a PR to docs/Hagzy_SuperAdmin_Documentation.pdf.